

REMOTE WIFI CONTROL OF CATAMARAN WITH T-100 THRUSTERS

OBJECTIVE

Enable real-time, wireless control of a catamaran equipped with two T-100 thrusters using a Raspberry Pi as the onboard controller and a remote computer as the operator interface. Communication will occur over a WiFi network, with keyboard-based teleoperation from the remote computer.

SYSTEM ARCHITECTURE

- **Raspberry Pi (on Catamaran)**
 - Runs a Python server script.
 - Controls two T-100 thrusters via ESCs using pigpio (PWM).
 - Receives commands over WiFi from the remote computer.
- **Remote Computer (Operator)**
 - Runs a Python client script.
 - Captures keyboard input (including arrow keys and space bar).
 - Sends control commands to the Raspberry Pi over WiFi.
- **WiFi Network**
 - Both devices must be connected to the same WiFi network.
 - Communication uses TCP sockets.

SOFTWARE COMPONENTS

Component	Platform	Purpose
pigpio daemon (pigpiod)	Raspberry Pi	Enables PWM control of GPIO pins
Server Script	Raspberry Pi	Receives commands, sets thruster PWM
Client Script	Remote Computer	Captures keyboard, sends commands
pynput library	Remote Computer	Captures special keys (arrows, space)

CONTROL MAPPING

Key/Command	Action
w	Both thrusters forward
s	Both thrusters backward
a	Rotate left (differential thrust)
d	Rotate right (differential thrust)
Arrow Up	Increase right thruster speed
Arrow Down	Decrease right thruster speed
Arrow Left	Decrease left thruster speed
Arrow Right	Increase left thruster speed
Space	Pause/Resume (set both to neutral)
x	Emergency stop (set both to neutral)
r	Reset both thrusters to neutral
q	Quit client and stop control

SETUP STEPS

1. Hardware

- Mount Raspberry Pi, ESCs, and T-100 thrusters on the catamaran.
- Connect ESCs to Pi GPIO pins (17 and 18).
- Provide separate power supplies for Pi and thrusters.

2. Software on Raspberry Pi

- Install pigpio (sudo apt install pigpio).
- Start pigpio daemon (sudo pigpiod).
- Copy and run the server script.

3. Software on Remote Computer

- Install Python and pynput (pip install pynput).
- Copy and run the client script.
- Set the Pi's IP address in the client script.

4. Network

- Ensure both devices are on the same WiFi network.

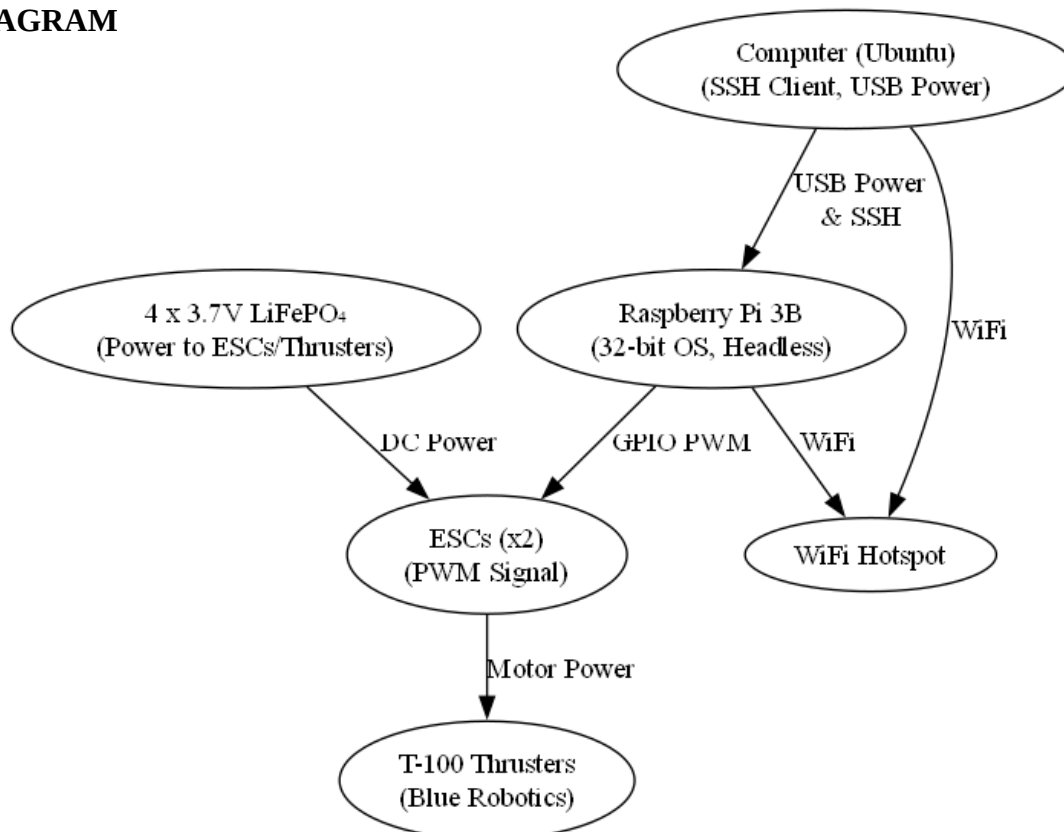
5. Operation

- Use keyboard to control the catamaran remotely in real time.

SAFETY NOTES

- Test with propellers removed for initial validation.
- Always start the system with thrusters in neutral.
- Ensure reliable WiFi coverage in the operating area.

DIAGRAM



SOFTWARE STACK OVERVIEW

- **pigpio:** A daemon and Python library for precise PWM and GPIO control on Raspberry Pi. It enables accurate servo/ESC (Electronic Speed Controller) control, which is critical for thruster operation.
- **Python:** Used for both server (Raspberry Pi) and client (remote computer) scripts, providing flexibility and ease of development.
- **pynput:** A Python library on the client side to capture keyboard events, including special keys like arrows and space bar.

HOW THE CODE WORKS

1. Server Script (Raspberry Pi)

Purpose:

Listens for commands over WiFi, translates them into PWM signals, and controls the two T-100 thrusters via ESCs.

Key Components:

- **pigpio Initialization:**
The script starts by connecting to the pigpiod daemon, which must be running on the Raspberry Pi (sudo pigpiod). This allows the script to send PWM signals to GPIO pins with high precision.
- **PWM Control Logic:**
The ESCs for the T-100 thrusters are connected to two GPIO pins (e.g., 17 and 18). The script sets their PWM pulse widths between 1100–1900 microseconds (μ s) to control speed and direction (forward, backward, and neutral).
- **Socket Server:**
The script opens a TCP socket and listens for incoming connections. Each command received (e.g., 'w', 'a', 'up', ' ') is mapped to a specific thruster action (increase/decrease PWM, pause, emergency stop, etc.).
- **Threading:**
Each client connection is handled in a separate thread, allowing multiple commands to be processed without blocking.
- **Logging:**
Every PWM change is logged with a timestamp for traceability and debugging.
- **Cleanup:**
On exit, the script sets both thrusters to neutral and releases the GPIO resources, ensuring the system stops safely.

2. Client Script (Remote Computer)

Purpose:

Captures keyboard input (including arrow keys and space bar), sends corresponding commands to the Raspberry Pi server over WiFi.

Key Components:

- **pynput Listener:**
Captures both standard keys (w, a, s, d, etc.) and special keys (arrow keys, space bar) in real time. This allows for intuitive, game-like control.
- **Socket Connection:**
For each keypress, the client opens a TCP connection to the Pi, sends the command, and prints the Pi's response (current thruster status).
- **Command Mapping:**
 - w: Move forward (both thrusters increase PWM)
 - s: Move backward (both decrease PWM)
 - a/d: Rotate left/right (differential thrust)
 - Arrow keys: Fine-tune individual thruster speeds
 - Space: Pause/resume (set both to neutral)
 - x: Emergency stop (neutralizes both)
 - r: Reset to neutral
 - q: Quit client

Why pigpio and Not RPi.GPIO?

- **Precision:** pigpio offers more precise PWM control than RPi.GPIO, which is essential for ESCs and thrusters.
- **PWM Range:** pigpio allows custom pulse widths, matching the requirements of most ESCs and servos.
- **Remote/Network Control:** pigpio can be used for remote GPIO control, but in this project, it's used locally on the Pi for direct hardware access.

How GPIO and PWM Work in This Context

- **GPIO Pins:** The Pi's GPIO pins can be set to output PWM signals, which the ESCs interpret as speed/direction commands for the thrusters.
- **PWM (Pulse Width Modulation):**
 - 1500µs = Neutral (no movement)
 - 1100–1500µs = Reverse (increasingly negative thrust)
 - 1500–1900µs = Forward (increasingly positive thrust)
- **Safety:** The code always returns to neutral on pause, emergency stop, or exit, preventing accidental runaway thrusters.

Summary Table: Code & Software Roles

Component	Technology	Function
pigpio	C daemon, Python lib	Accurate PWM for ESC/thruster control
Python server	Python	Receives commands, sets Pi GPIO/PWM
Python client	Python, pynput	Captures keyboard, sends commands over WiFi
TCP sockets	Python stdlib	Network communication between client & server

Software Flow Diagram

